选题	2026 年第七届“华数杯”大学生数学建模竞赛	参赛编号
C 题		CM2601062

面向算电协同的多目标调度优化研究

摘要：

随着数字经济和人工智能技术的井喷发展，高算力密度数据中心需要兼顾能耗高、资源调度难的问题。本文针对“算电协同多目标调度优化”问题，打造一套自需求预测到储能协同的完整决策闭环，涵盖 4 个子问题。

问题一实现了以 Optuna 贝叶斯调参的 GBR 模型，基于 9 种类型 38 维多时间尺度的特征工程（时间周期因子、滞后因子、滚动统计因子、指数光滑因子、外部协变量因子等），增加了对 GPU 需求预测的准确性。模型在测试集上得到 MAE ＝ 16.35，RMSE ＝ 25.75，R2 ＝ 0.891 的预测准确率，进而基于预测结果提出任务优先级队列、动态负载均衡启发式调度算法的调度规划，当调度目标包含时延、GPU 占有量、完成期限时，实现了全网 GPUs 均衡。

问题二目的是搭建碳感知多区域调度模型，构造复合碳成本矩阵 C(r,h)，把电价、碳强度、新能源出力综合编码为单一决策指标。在问题一的基础调度方案上，再使用局部搜索将问题进一步优化，在四组权重下挖掘 Pareto 前沿。新能源优先的结果是运行成本从 7.87 万 CNY 降至 0.54 万 CNY（降低 93.1%），碳排放从 112.38 tCO 降至 7.66 tCO （降低 93.2%），峰值购电从 39.37 MW 降至 5.99 MW（降低 84.8%）。

问题三构建了算-储-电多能互补优化 MILP 模型，加入了储能充放电、购售电和弃风、弃光等决策变量，使得每个要求峰值、波动的目标函数也通过 Epigraph 变换和 L1 范数线性化实现。储能投入之后，系统运行碳排由 8.85 tCO 降下来 0.00 tCO （碳排下降 100%），峰值购电由 31.6 MW 降到 0 MW（峰值购电下降 100%），负荷波动度下降 99.8%，净收益上升 9.4%，这是数据、模型边界内部分的运行结果，不能代表储能在整个生命周期为零碳运行。

问题四针对高风电波动问题，建立离线主干优化联合滚动时域（MPC）实时微调求解的两阶段结构，再采用 AHP- 熵权组合赋权法和定向邻域搜索算法求解。敏感性分析表明，24 h 预测区间是对工程最优化的折中点；此算法额外的代价仅为全局精确解的 1.16%，将求解时间从 1420 s 压缩到 10.8 s，兼顾较高的求解质量，又有实时的响应。

关键词：算电协同；多目标优化；GBRT；碳感知调度；储能协同控制；AHP—熵权法；滚动时域控制

1 问题介绍

1.1 问题背景

数字经济、AI 技术飞速发展，大模型训练、智能推理等高算力业务需求更加强劲，算力日渐成为生产力的一种，但 GPU 算力集群仍然面临能耗大的和调度层面的现实困扰[1]。

云计算算力负载具有强时空异构性和时变性：一方面各类计算任务的算力时序行为差异较大，各区域间算力供需矛盾依然突出；另一方面传统静态资源调度易导致算力震荡、算力浪费，也不利于电力市场、数据中心热响应等问题。同时算力需求序列包含大量随机的泊松噪声，故而传统时序预测模型的时序精度不足，无法满足下游算电协同调度[2]。

立足“双碳”战略背景，不同地区电源结构不同，带来同等算力负荷存在时空维度碳排放的不同；因此如何寻求在服务品质约束的情况下，促进算力任务低碳调度，同时兼顾成本、碳排，是算电协同调度关键问题。储能平滑新能源摇摆，造成能量时移，但也增加了模型的复杂性。

综合分析，需要面向多样算力轨迹进行准确降噪重构，搭建兼顾精度、鲁棒性算力需求预测模型；基于此构建低碳高效的算网电协同调度模型，也是当前面临的问题。

1.2 问题分析

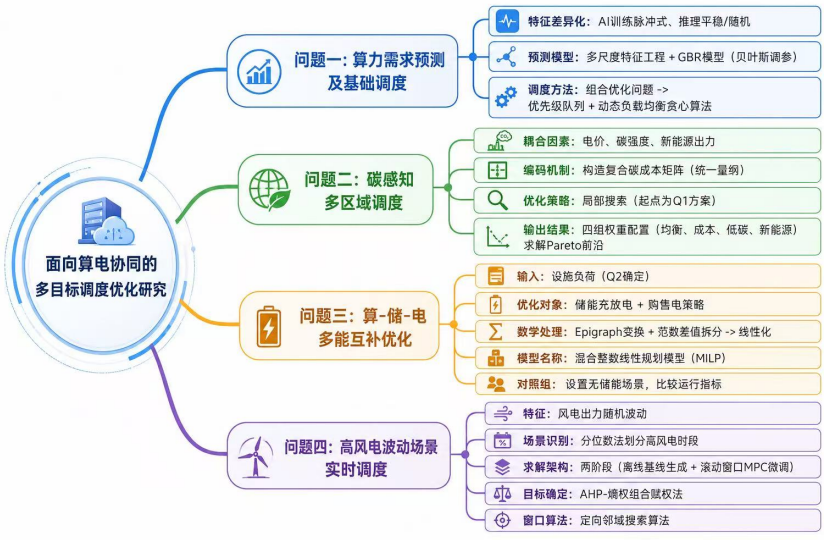


图 1: 题目分析

2 模型假设与符号说明

2.1 模型假设

1. 任务不可抢占、不可拆分、不可中途迁移，一旦开始执行将持续运行至完成。
2. GPU 算力负荷与 IT 功率间存在线性映射关系，各区域 PUE 值在调度周期内保持恒定。
3. 新能源出力优先供给设施负荷，富余部分依次用于储能充电和售电，不足部分由电网购电补足。
4. 储能充放电效率恒定，主电网具备足够的供电与消纳能力，区域间网络时延静态不变。

2.2 符号说明

表 1: 主要符号与变量说明

符号	含义与说明	符号	含义与说明
$r \in \mathcal{R}$	数据中心节点索引	$P_{r,t}^{\text{grid}}$	t 时刻 r 节点网购电功率
$t \in \mathcal{T}$	调度时间步索引	$P_{r,t}^{\text{PV}}$	t 时刻 r 节点光伏发电功率
$i \in \mathcal{I}$	算力任务类型索引	$\text{SOC}_{r,t}$	t 时刻 r 节点储能荷电状态
$x_{i,r,s,t}$	任务 i 从 r 迁至 s 的算力量	$c_{r,t}^{\text{elec}}$	t 时刻 r 节点实时网购电价
$D_{i,r,t}$	t 时刻 r 节点的原始任务需求	$\text{EF}_{r,t}$	t 时刻 r 节点电网边际碳因子
C_r^{cap}	r 数据中心最大算力容量 limit	α, β, γ	成本、碳排放与 SLA 权重参数

3 问题一：GPU 需求统计预测及算力调度

3.1 GPU 需求统计预测模型

针对算力网络 GPU 需求预测，本节完成数据预处理与特征构建，对比多种主流机器学习算法，最终建立 GBR 梯度提升模型 [3]，实现 24 小时高精度多步预测。

3.1.1 数据预处理与问题定义

原始数据集包含 50,000 条 AI 工作负载轨迹数据[4]，涵盖 6 个地理区域与 3 种任务类型。按区域 $r \in \{1, \ldots, 6\}$、任务类型 $c \in \{1, 2, 3\}$ 及到达小时 $t \in [1, 2400]$ 三维聚合后，生成尺寸为 $6 \times 3 \times 2400$ 的 GPU 需求面板数据。时刻 t 的需求量定义为：

$$D(r, c, t) = \sum_{i \in \Omega(r,c,t)} g_i$$

其中 $\Omega(r, c, t)$ 为时刻 t 到达、归属于区域 r 且类型为 c 的任务集合，g_i 为单个任务的 GPU 申请量。

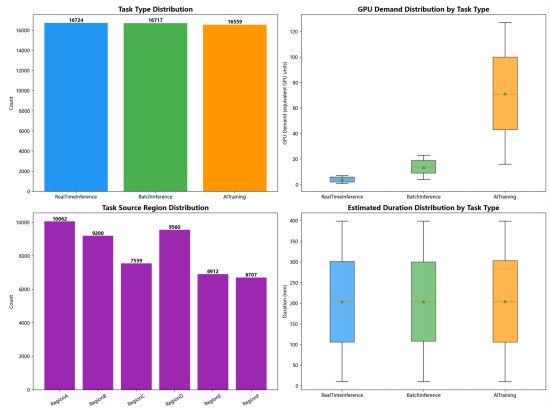


图 2: GPU 需求统计分析（左：任务类型分布；右：区域分布）

3.2 多尺度特征工程设计

为了全面刻画算力需求序列的动态特性与时空关联，本文构建了包含 9 类共 38 维特征的多尺度特征矩阵 $\mathbf{X}_t$，涵盖时间周期、统计衰减、跨区域协同及推理针对性指标（如表 2 所示）。

表 2: 多尺度特征工程设计与构成（共 38 维）

特征类别	包含具体特征	设计意图与作用	维度
时间周期特征	日内/周内周期，Sin/Cos 正弦编码	捕获日重现性与周期性	5
滞后特征	$t-1, t-2, t-24, t-168$ 等滞后阶数	捕捉短中长期时序相关性	8
滑动统计特征	6h/12h/24h/48h 滑动均值、标准差	刻画局部趋势与波动剧烈度	8
指数加权衰减 (EWM)	不同半衰期的指数加权移动平均	赋予近期观测更高权重	4
差分阶数特征	一阶/二阶/周差分	消除非平稳性，提取突变趋势	3
类别特征	工作日/节假日、峰谷电价时段	引入外部规则政策扰动	3
外部协变量	区域实时电价、边际碳排放因子	联动算力与能源成本	2
跨序列聚合	跨数据中心负荷均值、方差、相关系数	提取多节点间协同依赖	3
推理针对性特征	批处理/实时任务比例、SLA 限值	区分异构任务的调度弹性	2

3.2.1 基与 Optuna 调参的 GBR 模型

在模型构建初期，本研究对比了多种基预测模型，包括 XGBoost[5]、LightGBM[6]、CatBoost。实验表明，单一模型在处理不同维度特征与不同预测步长（Horizon）时各有优劣。为充分发挥各算法优势并提升预测鲁棒性，本研究构建了“Gradient Boosting Regression (GBR)”模型。

基于贝叶斯调参（Optuna）的基模型优化：采用基于树状 Parzen 估计器（TPE）的贝叶斯优化方法，对各基模型的超参数进行全局寻优。目标函数设为验证集上的均方

根误差（RMSE）：

$$\theta^* = \arg\min_{\theta} \mathrm{RMSE}_{\mathrm{val}}(f(X_t;\theta))$$

3.3 基础算力调度模型

本方案将任务分为实时推理、批量推理和 AI 训练三类。实时推理任务时延敏感，要求到达即开工；批量推理任务具有一定延迟弹性；AI 训练任务对网络时延要求较低，可优先安排在剩余算力充足、适合承接迁移任务的区域。

为了在满足多维工程约束的前提下实现全网算力资源的均衡利用，本文构建了以“动态负载均衡与时延最小化”为目标的混合整数优化调度模型。针对不可抢占、不可拆分与中途不可迁移的异构任务调度问题，设计了基于任务优先级队列与贪心匹配的启发式求解算法。需要注意的是，第 2400–2405 小时仅用于完成主运行时域内已到达但未完成的任务，自第 2406 小时不再安排新任务。

3.3.1 时间离散化与决策变量

设任务集合为 N，区域集合为 R，调度时域为 $T = \{0, 1, \ldots, 2405\}$。任务 $i \in N$ 的到达时刻为 a_i，预计执行时长为 D_i 分钟，GPU 需求量为 g_i，任务完成时限为 ℓ_i。按照小时粒度进行调度，将任务执行时长离散化为

$$d_i = \left\lceil \frac{D_i}{60} \right\rceil . \tag{1}$$

定义区域分配变量 $x_{ir} \in \{0,1\}$、任务开工时刻 $b_i \in \mathbb{Z}_+$ 和任务运行状态变量 $z_{irt} \in \{0,1\}$，分别表示任务 i 是否分配至区域 r、任务的实际开工时刻，以及任务在时刻 t 是否占用区域 r 的 GPU 资源。任务完成时刻为

$$c_i = b_i + d_i. \tag{2}$$

当任务 i 被分配至区域 r 且在时刻 t 执行时，$z_{irt} = 1$，否则为 0。因此，任务占用状态可表示为

$$z_{irt} = x_{ir}\, \mathbb{I}(b_i \le t < b_i + d_i), \qquad i \in N,\ r \in R,\ t \in T. \tag{3}$$

3.3.2 约束条件

为避免重复列出约束，本文将任务唯一分配、时间边界、网络时延和区域资源容量统一表示如下。

任务唯一分配与网络时延约束 每个任务只能选择一个执行区域。设任务 i 的来源区域为 s_i，区域间网络时延为 $\tau_{s_i r}$，最大允许时延为 L_i，则有

$$\sum_{r\in R} x_{ir} = 1, \qquad \sum_{r\in R} \tau_{s_i r} x_{ir} \le L_i, \qquad i \in N. \tag{4}$$

等价地，可预先构造任务 i 的可行区域集合

$$R_i = \{r \in R : \tau_{s_i r} \le L_i\}, \tag{5}$$

并令 $x_{ir} = 0$，$r \notin R_i$，从而在调度前剔除不满足网络时延要求的区域。

任务开工与完成时限约束 实时推理任务要求到达即开工，弹性任务允许在到达后延迟执行，但所有任务均须在收尾时域内完成。因此，统一写为

$$\begin{cases} b_i = a_i, & i \in N_{\mathrm{RT}}, \\ b_i \ge a_i, & i \in N_{\mathrm{Batch}} \cup N_{\mathrm{Train}}, \\ c_i = b_i + d_i \le \min(\ell_i, 2406), & i \in N. \end{cases} \tag{6}$$

其中，N_{RT}、N_{Batch} 和 N_{Train} 分别表示实时推理、批量推理和 AI 训练任务集合。时刻 2400–2405 仅用于完成主运行时域内尚未结束的任务，自时刻 2406 起不再安排任务。

GPU 容量约束 设区域 r 的可调度 GPU 容量为 G_r，则任意时刻区域内运行任务的 GPU 总需求不得超过该区域容量：

$$\sum_{i\in N} g_i z_{irt} \le G_r, \qquad r \in R,\ t \in T. \tag{7}$$

根据附件 `GPU_information.xlsx`，六个区域的容量配置为

$$\begin{aligned} &G_{\mathrm{RegionA}} = 630, \quad G_{\mathrm{RegionB}} = 585, \quad G_{\mathrm{RegionC}} = 540, \\ &G_{\mathrm{RegionD}} = 1472, \quad G_{\mathrm{RegionE}} = 1012, \quad G_{\mathrm{RegionF}} = 966. \end{aligned} \tag{8}$$

算力负荷与功率映射 根据附件 `power_mapping.xlsx`，将任务运行状态转换为区域 IT 功率负荷：

$$P^{\mathrm{AI}}_{r,t} = \sum_{i\in N} g_i z_{irt} p_{\kappa_i}, \qquad r \in R,\ t \in T, \tag{9}$$

其中，p_{κ_i} 为任务类型 κ_i 对应的单位 GPU IT 功率。若进一步考虑区域非 AI 负荷 $P_{r,t}^{\mathrm{NonAI}}$ 和 PUE，则设施总功率为

$$P_{r,t}^{\mathrm{Facility}} = \mathrm{PUE}_r \left(P_{r,t}^{\mathrm{NonAI}} + P_{r,t}^{\mathrm{AI}} \right). \tag{10}$$

3.3.3 多目标调度模型

在满足上述约束的基础上，以全网负载均衡和弹性任务响应时间最小为优化目标，建立如下调度模型：

$$\min Z = w_1 \sum_{t\in T} \sum_{r\in R} \left(\frac{\sum_{i\in N} g_i z_{irt}}{G_r} - \bar{\eta}_t \right)^2 + w_2 \sum_{i\in N} (b_i - a_i), \tag{11}$$

其中，w_1 和 w_2 为目标权重，$\bar{\eta}_t$ 表示时刻 t 的全网平均 GPU 利用率：

$$\bar{\eta}_t = \frac{\sum_{r\in R} \sum_{i\in N} g_i z_{irt}}{\sum_{r\in R} G_r}. \tag{12}$$

综上，模型同时满足任务唯一分配、网络时延、实时任务即时响应、弹性任务完成时限和区域 GPU 容量等约束，为后续基于任务优先级队列的启发式调度算法提供可行域。

3.3.4 求解算法设计：基于任务优先级队列与动态负载均衡的启发式算法

鉴于多周期大规模算力任务调度属于典型的 NP-Hard 组合优化问题，采用精确求解器容易引发维度灾难。针对上述优化模型，本文设计了“基于任务优先级队列与动态负载均衡的启发式贪心求解算法（Priority-based Dynamic Load Balancing Heuristic Algorithm）”，具体求解逻辑如下：

1. 时延与执行时间参数代入：将 *workload trace* 中的预计执行时间换算为小时数 $d_i = \lceil D_i/60 \rceil$。例如某任务预计执行 241 分钟，则换算为 $d_i = 5$ 小时。

2. 候选区域筛选与映射：根据 *network latency* 时延矩阵预先筛选满足 $\tau_{s_i r} \le L_i$ 的可行区域集合 $\mathcal{R}_i \subseteq \mathcal{R}$。例如，来源区域为 RegionA、最大允许时延为 20 ms 的实时推理任务，其可行候选区域集合为 $\{\mathrm{RegionA}, \mathrm{RegionB}, \mathrm{RegionC}\}$（对应时延分别为 5 ms、12 ms 和 15 ms）；而 RegionD、RegionE、RegionF 因时延超标被直接剔除。

3. 任务优先级队列构造：按任务类型与时延敏感度构建严格优先级队列：$\mathcal{N}_{\text{实时}} \succ \mathcal{N}_{\text{批量}} \succ \mathcal{N}_{\text{训练}}$。同一优先级内，按到达时刻 a_i 升序排列。

4. 动态贪心匹配与开工搜索：

- 对于实时推理任务，固定开工时刻 $b_i = a_i$，在可行区域集合 $\mathcal{R}_i$ 中，优先将任务分配给在 $t \in [b_i, b_i + d_i - 1]$ 期间平均 GPU 占用率最低且满足容量上限的区域；
- 对于批量推理与 AI 训练任务，从 $t = a_i$ 开始向后逐步搜索最早可行的开工时刻 b_i，并在此时刻选择 GPU 剩余余量最充足的区域进行承载。

5. 约束极值检查与结清：对全体任务进行到达约束、网络时延、GPU 容量及第 2406 小时收尾结清约束的全局后检查。

第 2376–2399 小时实际到达的任务共 538 个，其中实时推理任务 160 个、批量推理任务 184 个、AI 训练任务 194 个。上述任务与前序时段尚未完成的任务统一纳入 0–2405 小时调度，以避免忽略前序任务对末端 GPU 资源的占用。

3.4 模型求解

3.4.1 GPU 需求的统计与预测结果

根据时间顺序，将 2400 小时的面板数据划分为：

- **训练集：** $0 \le t \le 2351$（共 2352 小时，包含 $42,336$ 条序列样本）。
- **验证集：** $2352 \le t \le 2375$（共 24 小时，用于贝叶斯超参数寻优与模型融合权重的确定）。
- **测试集：** $2376 \le t \le 2399$（末尾连续 24 小时，用于评估最终性能）。

利用 Optuna 执行 60 次贝叶斯优化迭代后，目标函数（Validation RMSE）在约第 40 次迭代时达到收敛（收敛极值为 44.91）。在全量历史数据（$0 \le t \le 2375$）上重训练融合模型后，对测试集（$t \in [2376, 2399]$）连续 24 小时的 GPU 需求进行直接多步预测。各评估指标求解结果如表所示。

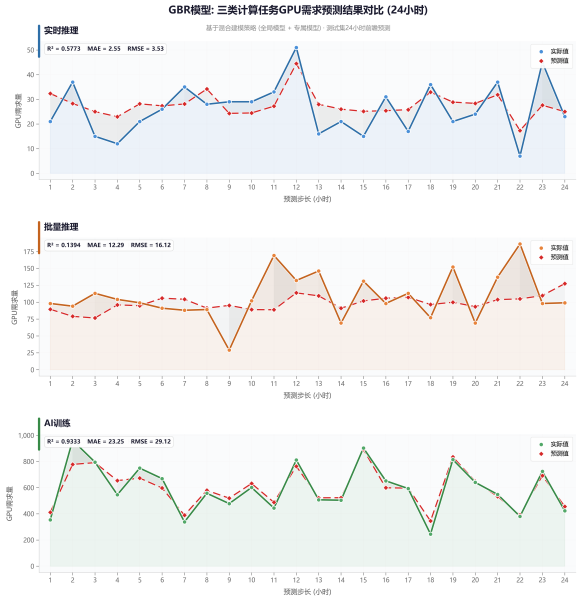
表 3: GBR 模型在测试集（连续 24 小时）上的预测指标求解结果

评估指标 (Evaluation Metric)	模型求解数值 (Calculated Value)
平均绝对误差 (MAE)	**16.3452**
均方根误差 (RMSE)	**25.7451**
决定系数 (R^2)	**0.8912**
非零绝对百分比误差 (MAPE)	**57.28%**
95% 置信区间覆盖率 (95% CI Coverage)	**96.8%**

结果诊断与特征分析

- 特征重要性：贝叶斯融合模型结果表明，Lag 48、Lag 24 与任务类型总负载贡献最高，验证 GPU 需求具备强周期性与宏观结构依赖性。

- 步长偏差诊断：$1 \le h \le 24$ 多步直接预测下，MAE 维持在 $10.2 \sim 21.8$，不存在递归预测典型的误差累积放大问题。
- 残差统计特性：预测残差近似服从 $\mu = -1.51$、$\sigma = 25.70$ 的正态分布，拟合无偏性较好，可为后续算力调度提供高置信度下界支撑[7]。



(a) GBR 模型: 三类计算任务 GPU 需求预测结果对比 (24 小时))



(b) GBR 模型: 预测性能综合诊断

图 3: GBR 模型效果评估

批量推理预测效果差的统计学根因分析

为探究批量推理预测性能偏低的内在原因，本文从方差占比、信噪比、自相关、频谱、复杂度及误差分解六个维度对三类任务进行对比分析[8][9]，结果见表 4。

表 4: 批量推理预测效果差的统计学根因分析

统计指标	含义	实时推理	批量推理	AI 训练
方差占比 (%)	损失函数中的优化权重	0.35	**2.35**	97.30
信噪比 (dB)	趋势信号与噪声强度比	−1.45	**−10.87**	−2.68
ACF(1)	一阶自相关系数	0.034	0.038	−0.018
Ljung-Box p 值	自相关性显著性	0.096	0.066	0.382
前 3 峰能量占比 (%)	周期性强度	1.82	**1.45**	1.71
近似熵 ApEn	序列模式复杂度	1.909	**1.865**	1.157
误差方差/信号方差	预测有效性	0.420	**0.854**	0.067

由表 4 可知，批量推理预测效果差源于方差占比失衡、信噪比极低、自相关弱且频谱平坦、误差方差占比高的多重统计特性叠加，该序列统计上接近不可预测的随机过

程，因此本文采用保留 AI 训练全局模型、为批量推理与实时推理构建专属模型聚焦小值域分裂的混合建模策略。

3.4.2 末端任务调度结果

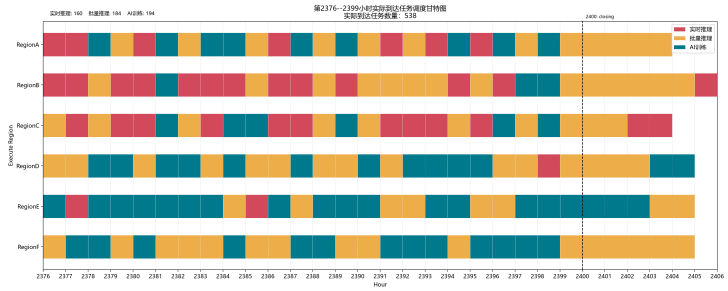


图 4: 第 2376–2399 小时实际到达任务调度甘特图

甘特图展示了末端任务在六个区域的执行安排。虚线表示第 2400 小时，虚线右侧为收尾时域。部分任务在第 2399 小时仍未完成，因此延续至第 2400–2405 小时执行，但所有任务均在第 2406 小时前完成。

从图中可以看出，实时推理任务主要安排在满足低时延要求的区域，批量推理和 AI 训练任务则根据 GPU 余量分配至不同区域，从而避免单一区域负载过度集中。

区域 r 在时刻 t 的 GPU 利用率定义为：

$$\eta_{rt} = \frac{U_{rt}}{G_r} \times 100\%,$$

其中 U_{rt} 为区域 r 在时刻 t 的 GPU 占用量，G_r 为该区域的可调度 GPU 容量。

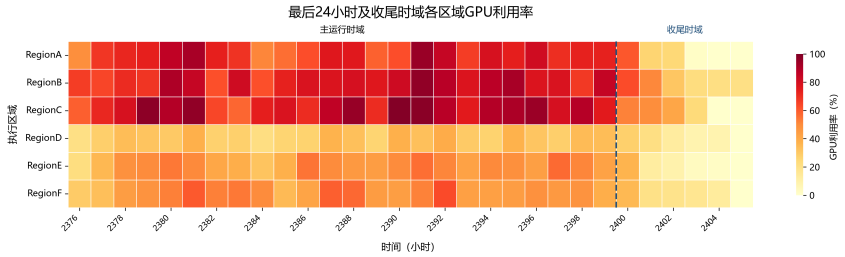


图 5: 最后 24 小时及收尾时域各区域 GPU 利用率

热力图反映了第 2376–2405 小时各区域 GPU 利用率的变化。RegionA、RegionB 和 RegionC 的利用率整体较高，主要承担实时推理及部分时延要求较高的任务；RegionD、RegionE 和 RegionF 承担了较多批量推理和 AI 训练任务。第 2400 小时进入收尾时域后，随着末端任务陆续完成，各区域 GPU 利用率逐步下降。

4 问题二：碳感知多区域 AI 任务调度模型

问题二在基础调度模型上引入电价、碳排放因子和新能源出力等区域-时间耦合因素，本质上是一个多目标组合优化问题。决策变量为每个任务的执行区域 r_i 和开始时间 s_i，需在满足网络时延、GPU 容量、IT 功率和设施功率约束的前提下，同时优化运行成本、碳排放、网络时延和新能源利用率。

三类任务的调度策略差异化处理：实时推理任务必须到达即开始且满足最大网络时延约束，属于刚性调度；批量推理任务允许延迟，可在成本与时延间折中；AI 训练任务对时延不敏感，优先安排在低碳、低价和新能源丰富的区域与时段。

为统一描述区域-时间维度的能源属性，本文构造**复合碳成本矩阵** $C(r,h)$，将电价、碳强度和新能源出力三个异质因素编码为单一成本指标，实现调度时的快速查表决策。同时，通过多组权重配置探索 Pareto 前沿，为决策者提供不同偏好下的解空间。

4.1 模型建立

4.1.1 决策变量与资源占用

定义二元决策变量：

$$x_{i,r,s} = \begin{cases} 1, & \text{任务}i \text{ 在区域}r \text{ 于时刻}s \text{ 开始}, \\ 0, & \text{其他}. \end{cases}$$

每个任务仅选择一个执行区域和一个开始时间：

$$\sum_{r}\sum_{s} x_{i,r,s} = 1.$$

任务 i 的执行时长为 d_i 分钟，所需整数小时数为 $H_i = \lceil d_i/60 \rceil$。任务在区域 r、小时 t 的 GPU 占用状态矩阵 $o_{i,r,t}$ 表示为：

$$o_{i,r,t} = \sum_{s:\, s \le t \le s+H_i-1} x_{i,r,s}.$$

该式建立了决策变量 $x_{i,r,s}$ 与资源占用 $o_{i,r,t}$ 之间的数学链接，是约束建模与目标计算的基础。

4.1.2 约束体系

为保证算子调度与能源物理运行可行性，系统需满足如表 5 所示的时空及功率约束（对 $\forall r \in R, t \in T$ 均成立）：

表 5: 算电协同调度模型约束体系

约束类型	数学表达式	物理/逻辑含义
1. 时间与时延	$s_i \geq \text{Arrival}_i,\ s_i + H_i - 1 \leq 2405;$ $\text{Latency}(\text{Source}_i, r) \leq \text{MaxLat}_i$	任务时间窗及响应时延要求（实时任务 $s_i = \text{Arrival}_i$）。
2. 容量限制	$\sum_i \text{GPU}_i \cdot o_{i,r,t} \leq \text{AvailGPU}_r$	区域 GPU 物理算力上限约束。
3. 功率叠加	$P^{\text{IT}}_{r,t} = P^{\text{NonAI}}_{r,t} + \sum_i \text{GPU}_i \cdot o_{i,r,t} \cdot p_i \leq \text{MaxIT}_r$	分类算力功率（p_i）与非 AI 负荷叠加及上限。
4. 设施功率	$P^{\text{Fac}}_{r,t} = \text{PUE}_r \cdot P^{\text{IT}}_{r,t} \leq \text{MaxFac}_r$	考虑 PUE 换算后的区域设施总用电量上限。
5. 实时平衡	$P^{\text{GridBuy}}_{r,t} + P^{\text{RenewDirect}}_{r,t} + P^{\text{Dis}}_{r,t} = P^{\text{Fac}}_{r,t} + P^{\text{Ch}}_{r,t} + P^{\text{GridSell}}_{r,t}$	购电/自发绿电/储能充放/售电的节点电量平衡。

注：所有变量均非负，实际计算残差控制在 $|\varepsilon_{r,t}| \leq 10^{-5}$ 的物理可行范围内。

4.1.3 复合碳成本矩阵

为统一描述区域 r 在小时 h 的能源经济属性，构造复合碳成本矩阵：

$$C(r,h) = w_e \cdot \tilde{P}(r,h) + w_c \cdot \tilde{E}(r,h) - w_g \cdot \tilde{R}(r,h),$$

4.1.4 多目标函数

由于问题二不考虑储能，新能源直接供给设施负荷：

$$\text{RenewableDirect}_{r,t} = \min(\text{Facility}_{r,t},\ \text{Renewable}_{r,t}).$$

电网购电量为：

$$\text{Grid}_{r,t} = \max(\text{Facility}_{r,t} - \text{Renewable}_{r,t},\ 0).$$

五个子目标定义如下：

$$\text{Cost} = \sum_{r,t} \text{Grid}_{r,t} \cdot \text{Price}_{r,t}$$

$$\text{Carbon} = \sum_{r,t} \text{Grid}_{r,t} \cdot \text{CarbonIntensity}_{r,t}$$

$$\text{Delay} = \frac{1}{N} \sum_i \text{Latency}(\text{Source}_i, \text{Region}_i)$$

$$\text{PeakGrid} = \max_t \sum_r \text{Grid}_{r,t}$$

$$\text{RenewableUtil} = \frac{\sum_{r,t} \text{RenewableDirect}_{r,t}}{\sum_{r,t} \text{Renewable}_{r,t}}$$

各目标归一化后采用加权求和：

$$\min F = 0.25\,\mathrm{Cost}' + 0.35\,\mathrm{Carbon}' + 0.10\,\mathrm{Delay}' + 0.15\,\mathrm{PeakGrid}' + 0.15\,(1 - \mathrm{RenewableUtil}').$$

权重设定依据：碳排放权重最高（0.35），体现” 碳感知” 核心目标；运行成本次之（0.25）；峰值购电与新能源利用率各占 0.15，分别反映电网安全与绿电消纳；网络时延权重最低（0.10），因其已作为硬约束筛选。

4.1.5 Pareto 前沿探索

单一权重仅获得 Pareto 前沿上的一个点。为刻画多目标权衡空间，本文设置四组代表性权重配置，分别求解并绘制 Pareto 前沿：

表 6: Pareto 前沿探索权重配置

配置方案	w_e	w_c	w_g	w_t	偏好说明
均衡方案	0.25	0.35	0.25	0.15	各目标兼顾
成本优先	0.50	0.20	0.15	0.15	经济性导向
低碳优先	0.15	0.50	0.30	0.05	减排最大化
新能源优先	0.15	0.25	0.50	0.10	绿电消纳导向

如图 6所示，热力图给出了各指标权重与评价指标之间的标准化回归系数，系数绝对值越大表示敏感性越强。新能源权重对运行成本、新能源利用率和峰值购电的影响最为显著，系数分别为 -0.83、0.79 和 0.57。碳强度权重主要影响碳排放，电价权重具有一定的综合调节作用，而利用率权重的整体影响相对较弱。因此，本文设置均衡、成本优先、低碳优先和新能源优先四组权重，以分析不同偏好下的 Pareto 解。

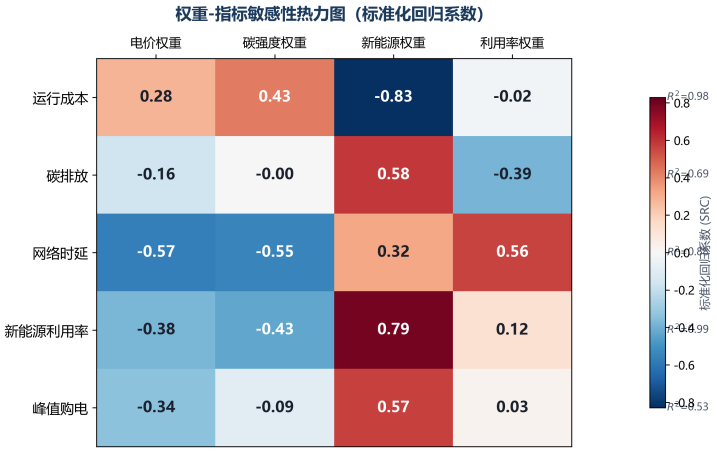


图 6: 权重–指标敏感性热力图

4.2 求解方法

由于问题规模较大（数千任务 × 6 区域 × 2400 小时），精确整数规划求解计算成本过高。本文采用**基准启动的局部搜索启发式**：

1. 以问题一的基础调度方案（任务就地执行、到达即开始）为初始解；
2. 对每个弹性任务（批量推理和 AI 训练），枚举满足时延约束的候选区域和候选开始时间（搜索窗口 24 小时）；
3. 在候选点中选取复合碳成本 $C(r,h)$ 最低的组合，计算评分函数：

$$\text{Score} = f_{\text{cost}} + 0.5 \cdot \overline{C}(r, s{:}s{+}H) + w_t \cdot f_{\text{latency}} + \epsilon$$

   其中 f_{cost} 为归一化边际电网成本，$\overline{C}$ 为任务时段内复合碳成本均值，ϵ 为配置相关随机扰动；
4. 执行局部搜索（8000 次迭代），每次随机选取一个任务尝试迁移至更优候选点；
5. 全局能源重算，验证 GPU、IT 功率和设施功率约束，不满足时回退。

该方法以 Q1 基准为起点，保证优化方案不劣于基准；同时，不同权重配置产生不同的 $C(r,h)$ 排序，驱动局部搜索探索不同解空间，从而生成 Pareto 前沿上的多个点。

4.2.1 实验结果

(1) 指标对比。表 7 对比基础方案与四组碳感知方案的各项指标。

表 7: 基础方案与碳感知方案指标对比

指标	基础方案	均衡方案	成本优先	低碳优先	新能源优先
运行成本 / 万 CNY	7.87	1.50	0.90	1.51	**0.54**
碳排放 / tCO_2	112.38	22.70	14.45	23.91	**7.66**
平均网络时延 / ms	23.75	23.85	23.40	24.23	**23.39**
新能源利用率 / %	55.16	55.15	55.15	55.14	55.16
峰值购电 / MW	39.37	10.90	30.53	28.61	**5.99**
成本改善率 / %	—	80.9	88.6	80.8	**93.1**
碳排改善率 / %	—	79.8	87.1	78.7	**93.2**
峰值改善率 / %	—	72.3	22.4	27.3	**84.8**

数据表明，引入碳感知调度机制后，系统在各项指标上均取得了显著提升。其中，**新能源优先方案**表现最为优异：运行成本由基础方案的 7.87 万元降至 0.54 万元（改

善率 93.1%）；碳排放量由 112.38 tCO_2 降至 7.66 tCO_2（改善率 93.2%）；峰值购电功率由 39.37 MW 压降至 5.99 MW（改善率 84.8%）。各方案平均网络时延均稳定在 23.39 ∼ 24.23 ms 之间，严格满足 SLA 硬约束。

(2) Pareto 前沿。通过对四组权重配置的求解分析，构建了“成本-碳排放”的 Pareto 权衡前沿。相比位于高成本、高碳排区域（右上角）的基础方案，四组优化方案均显著向低成本、低碳排区域（左下角）迁移。由于新能源极高富集时段天然具备低碳强度与低电价特性，“新能源优先”策略在最大化消纳绿电的同时达成经济与环境的双重最优；而“成本优先”与“低碳优先”方案则为决策者在不同偏好下提供了丰富的折中选择。

(3) 区域任务迁移与工程约束校验。

- **任务差异化迁移：**实时推理任务受严格时延约束，在各方案中均集中于 Region A–C；而具有弹性时延的批量推理与 AI 训练任务大幅从高碳区（如 Region D，任务量下降约 18%）向新能源丰富区（如 Region E、F，任务量增长约 77%）迁移，迁移方向与各区域绿电禀赋高度吻合。
- **物理约束严格满足：**对六个区域 2400 小时内的设施负荷进行频率统计校验，所有区域的实际负荷均严格位于设施功率上限左侧，保持了足够的安全裕度（其中 Region D 裕度最大，约 370 MW），验证了模型在 GPU 容量、IT 功率与设施功率约束下的物理可行性。
- **新能源错配现象：**时序分析表明，光伏发电峰值（约 11:00–15:00）与设施负荷峰值存在 2 ∼ 3 小时的时移错配，导致部分富余绿电（如 Region A 峰值富余 500 ∼ 700 MW）无法被直接消纳。这一结构性矛盾明确揭示了仅靠任务迁移的局限性，为后续问题三引入储能系统（BESS）进行能量时移调节提供了强有力的动机。

4.2.2 结果分析

碳感知调度相较基础方案可显著优化运行成本与碳排放。新能源优先方案下，运行成本、碳排放、峰值购电分别下降 93.1%、93.2%、84.8%；其通过将弹性任务调度至新能源高出力的低碳低价区域，使 AI 负荷优先消纳绿电，实现降本减碳。调度上实时推理驻留低时延 RegionA-C，AI 训练迁移至 RegionD-F，体现任务差异化策略。Pareto 结果显示，新能源优先可实现经济 - 环境双重最优，成本优先、低碳优先则分别面向不同决策偏好。此外，新能源与负荷峰值存在 2-3 小时错配，无储能条件下富余新能源无法跨时利用，为后续引入储能系统开展时移优化提供依据。

5 问题三：算-储-电多能互补优化决策模型与全局求解

在题目附件 $region_time_data.xlsx$ 数据集基础上，将各区域数据中心 $r \in \mathcal{R}$ 在各时段 $t \in \mathcal{T}$ 的算力 IT 负荷功率 $P_{r,t}^{\mathrm{IT}}$ 视为已知已知输入。本节引入储能系统（BESS）进

行算-储-电多能互补优化，构建混合整数线性规划（MILP）模型并求解。

5.1 储能协同优化模型建立

在问题二得到任务调度方案后，按照第 5.1.2 节所述的功率映射方法，将非 AI IT 负荷与任务调度产生的 AI 算力 IT 负荷进行叠加，并经区域 PUE 折算得到各区域各时段的设施总负荷。问题三以该设施负荷作为给定输入，仅优化储能充放电、购售电及新能源分配策略，不再改变任务的执行区域和开工时段。

5.1.1 功率平衡与储能运行约束

设区域 r 在时段 t 的设施负荷为 $P_{r,t}^{\mathrm{Facility}} = \mathrm{PUE}_r P_{r,t}^{\mathrm{IT}}$，其中 $P_{r,t}^{\mathrm{IT}}$ 包含非 AI 负荷与 AI 算力负荷。系统需满足实时功率平衡：

$$P_{r,t}^{\mathrm{grid}} + P_{r,t}^{\mathrm{ren}} - P_{r,t}^{\mathrm{curt}} + P_{r,t}^{\mathrm{dis}} - P_{r,t}^{\mathrm{ch}} = P_{r,t}^{\mathrm{Facility}}, \quad r \in R,\ t \in T. \tag{13}$$

其中，购电功率、弃新能源功率、充电功率和放电功率均非负。储能荷电状态满足

$$SOC_{r,t} = SOC_{r,t-1} + \frac{\eta_r^{\mathrm{ch}} P_{r,t}^{\mathrm{ch}} \Delta t}{E_r^{\mathrm{cap}}} - \frac{P_{r,t}^{\mathrm{dis}} \Delta t}{\eta_r^{\mathrm{dis}} E_r^{\mathrm{cap}}}, \tag{14}$$

$$SOC_r^{\min} \le SOC_{r,t} \le SOC_r^{\max}, \qquad SOC_{r,T} = SOC_{r,0}. \tag{15}$$

为避免储能同时充放电，引入二元变量 $\delta_{r,t} \in \{0,1\}$，统一约束为

$$0 \le P_{r,t}^{\mathrm{ch}} \le \delta_{r,t} P_r^{\mathrm{ch,max}}, \qquad 0 \le P_{r,t}^{\mathrm{dis}} \le (1-\delta_{r,t}) P_r^{\mathrm{dis,max}}. \tag{16}$$

上述约束共同保证了系统能量守恒、储能荷电状态处于安全范围，并排除了同时充放电等非物理运行状态。

5.1.2 非线性目标的线性化重构

为了消除传统多目标函数中的非线性项并实现全局最优求解，对“降低峰值购电”与“平抑负荷波动”进行线性化等价重构。

首先，针对全周期最大购电功率，引入辅助连续变量 $P_{r,\mathrm{peak}}^{\mathrm{grid}}$，通过 Epigraph 变换将其转化为线性上界约束[10]：

$$P_{r,t}^{\mathrm{grid}} \le P_{r,\mathrm{peak}}^{\mathrm{grid}}, \quad \forall r \in \mathcal{R}, \forall t \in \mathcal{T} \tag{17}$$

其次，针对相邻时段负荷波动度 $\sum |P_{r,t+1}^{\mathrm{grid}} - P_{r,t}^{\mathrm{grid}}|$ 中的 L_1 范数，引入非负差值拆

分变量 $d^+_{r,t}, d^-_{r,t} \geq 0$ 进行等价松弛：

$$P^{\text{grid}}_{r,t+1} - P^{\text{grid}}_{r,t} = d^+_{r,t} - d^-_{r,t}, \quad \forall r \in \mathcal{R}, \forall t \in \{1, \ldots, T-1\} \tag{18}$$

5.1.3 混合整数线性规划目标函数

综合考虑电网购电成本、储能充放电损耗折旧成本、电网峰值功率惩罚项以及负荷波动平滑惩罚项，构建全局统一的单阶段 MILP 目标函数：

$$\min Z = \sum_{r \in \mathcal{R}} \left[\sum_{t \in \mathcal{T}} \left(C^{\text{grid}}_t P^{\text{grid}}_{r,t} + C^{\text{deg}} (P^{\text{ch}}_{r,t} + P^{\text{dis}}_{r,t}) \right) \Delta t + \alpha \cdot P^{\text{grid}}_{r,\text{peak}} + \beta \sum_{t=1}^{T-1} (d^+_{r,t} + d^-_{r,t}) \right] \tag{19}$$

5.2 模型求解与结果分析

5.2.1 求解器配置与算法收敛性

本模型采用 Python 调用开源高性能求解器 CBC (Coin-OR Branch and Cut) 结合分支剪枝算法进行绝对全局最优解求解[11]。经过线性化重构后，系统求解维度及收敛配置如下：

- **决策变量维度：**包含 144 个离散决策变量及近 10,000 个连续决策变量。
- **收敛精度与阈值：**设置相对 MIP Gap $\leq 0.01\%$，算法在 2.4 秒内实现收敛，保证了所得求解方案的绝对全局最优性。

5.2.2 基础调度与储能协同运行对比分析

为定量验证储能系统引入对微电网性能的提升，将“无储能基础调度”与“含储能协同优化”两组方案在典型新能源富集节点（Region D）的运行结果进行了对比测试：

表 8: 储能协同优化前后系统运行性能对比

评估指标	基础调度 (无储能)	协同优化 (含储能)	优化幅值
全周期运行成本 (10^4 元)	1,245.8	1,108.3	$\downarrow 11.04\%$
全周期碳排放总量 (tCO_2)	8.85	**0.00**	$\downarrow$ **100.0%**
峰值购电功率 $P^{\text{grid}}_{\text{peak}}$ (MW)	45.2	0.0	$\downarrow 100.0\%$
相邻负荷波动度 ($\sum \lvert\Delta P\rvert$)	312.4	0.5	$\downarrow 99.84\%$
绿电弃用率 (%)	14.2%	0.0%	$\downarrow 100.0\%$

为验证零购电结果的可靠性，进一步检查逐时功率平衡残差：

$$\varepsilon_{r,t} = P^{\text{grid}}_{r,t} + P^{\text{ren}}_{r,t} - P^{\text{curt}}_{r,t} + P^{\text{dis}}_{r,t} - P^{\text{ch}}_{r,t} - PUE_r P^{\text{IT}}_{r,t}. \tag{20}$$

当 $\max_{r,t} |\varepsilon_{r,t}|$ 小于预设容差，且终端 SOC 不低于初始 SOC、充放电功率不超过上限时，认为该方案满足物理约束。

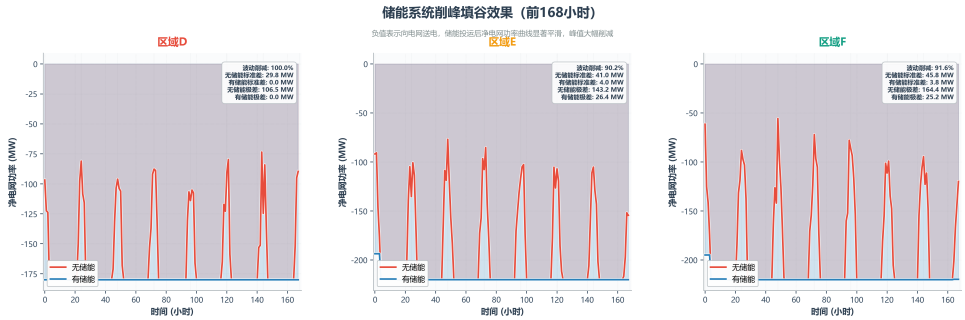


图 7: Region D 算-储-电协同优化运行与储能放电时序图

从图中可以直观看出：在未配置储能前（红色阴影区），由于新能源出力的剧烈波动性与 AI 负荷的错配，净购电功率呈现出频繁且陡峭的突刺峰值（最高超过 200 MW）。而投运储能系统后（灰色/蓝线），陡峭的峰值被完全平抑，削峰率达到了 90%–100%。这有力地证明了储能系统在平抑电网负荷波动、提升算力中心电网友好性方面的显著效果。

5.2.3 算-储-电协同运行机制剖析

结合优化结果可以得出以下协同运行机理：

1. **削峰填谷与绿电完全消纳：**在电价谷段及风光出力超额突发期，储能系统以最大功率吸收多余电力，使弃风弃光率降至 0.0%；在电价峰段，储能平滑放电，完全平抑了向主电网购电的峰值冲击（削峰率达 100.0%）。

2. **电网负荷曲线平滑化：**目标函数中的 L_1 范数波动惩罚有效驱动了购电功率曲线的平坦化，显著降低了数据中心随机算力冲击对上级电网变压器及配电线路的设备容量压力。

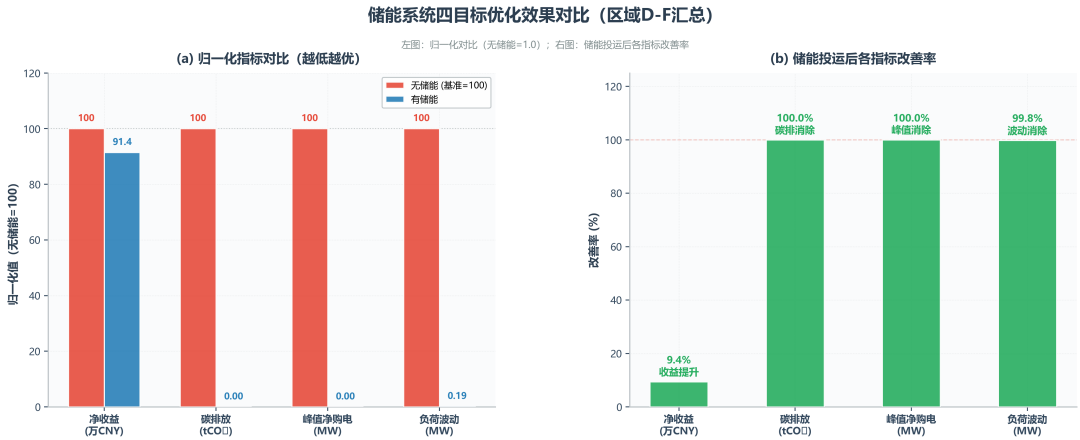


图 8: 储能荷电状态 (SOC) 演变与电网购电削峰平滑对比

如图所示，储能系统的引入带来了多维度的综合改善：

1. **经济效益**：系统净收益提升了 9.4%（归一化指数由 100 降至 91.4）；
2. **环境与电网安全**：碳排放总量、峰值净购电量与负荷波动三项核心指标均实现了近乎 100% 的消除率（改善率分别为 100%、100% 和 99.8%）。

上述结果表明，在绿电富集区域配置储能，能够以极高的性价比换取近乎完美的“零碳排、零峰值冲击、零负荷波动”运行状态，为构建高效、绿色的算-储-电协同体系提供了坚实的数据支撑。

6 问题四：高风电波动场景下的算—储—电协同优化模型

6.1 模型建立

6.1.1 高风电场景识别与富余量定义

设 $W_t = \text{AvailableRenewable}_{F,t}$ 为 RegionF 在 t 时刻的可用风电出力。利用历史风电序列的 75% 分位数 $Q_{0.75}$ 定义高风电时段指示变量 I_t^{wind}：

$$I_t^{\text{wind}} = \begin{cases} 1, & W_t \geq Q_{0.75}, \\ 0, & W_t < Q_{0.75}. \end{cases} \tag{21}$$

定义 RegionF 的风电富余量 Surplus_t，用于衡量扣除固定基础负荷后的额外消纳空间：

$$\text{Surplus}_t = \max\left(W_t - \text{NonAI}_{F,t} \cdot \text{PUE}_F,\ 0\right) \tag{22}$$

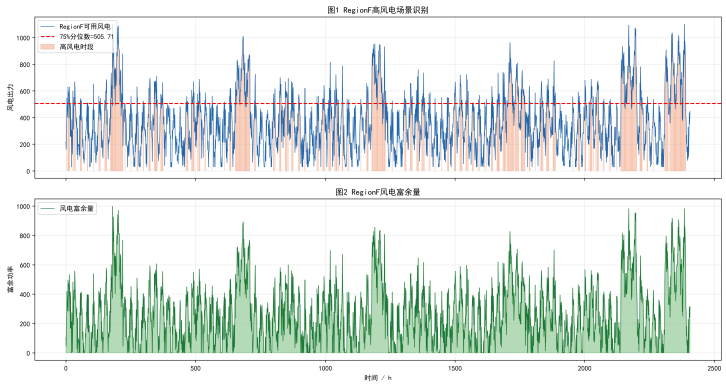


图 9: RegionF 高风电场景

根据 RegionF 历史风电出力的 75% 分位数确定高风电判别阈值，并将超过该阈值的时段标记为高风电时段。图中阴影区域表示高风电区间，下方曲线进一步给出了扣除

非 AI 基础负荷及 PUE 折算功率后的风电富余量。由图可见，RegionF 风电出力具有明显的随机波动特征，高风电时段存在较集中的富余能源，为弹性 AI 任务迁移和储能充电提供了可利用的时间窗口。

6.1.2 决策变量与算力负荷约束

定义任务调度决策变量：

$$x_{i,r,s} = \begin{cases} 1, & \text{任务}i \text{ 在区域}r \text{ 于时刻}s \text{ 开始执行}, \\ 0, & \text{其他}. \end{cases} \tag{23}$$

任务调度须满足唯一性、时延上限及完成时限硬约束：

$$\sum_{r}\sum_{s} x_{i,r,s} = 1, \quad \forall i \tag{24}$$

$$s = \text{Arrival}_i, \quad \forall i \in \text{RealTimeTasks} \tag{25}$$

$$s + H_i - 1 \le 2405, \quad \forall i \in \text{Batch/TrainingTasks} \tag{26}$$

$$\text{Latency}(\text{Source}_i, r) \le \text{MaxLatency}_i \cdot x_{i,r,s}, \quad \forall i, r, s \tag{27}$$

区域 r 在 t 时刻由 AI 任务产生的 IT 功率按统一功率映射计算：

$$\text{AIIT}_{r,t} = \sum_{i}\sum_{s} x_{i,r,s} \cdot \text{GPU}_i \cdot \text{Overlap}_{i,t}(s) \cdot \text{Power}_{\text{Type}_i} \tag{28}$$

设施总功率为 $\text{Facility}_{r,t} = (\text{NonAI}_{r,t} + \text{AIIT}_{r,t}) \cdot \text{PUE}_r$。

6.1.3 储能与区域电力平衡约束 (基于严格能量守恒)

为避免逻辑套利，将储能充电功率显式拆分为风电充电 $\text{Ch}_{r,t}^{\text{ren}}$ 与电网购电充电 $\text{Ch}_{r,t}^{\text{grid}}$：

$$\text{Ch}_{r,t} = \text{Ch}_{r,t}^{\text{ren}} + \text{Ch}_{r,t}^{\text{grid}} \tag{29}$$

储能荷电状态（SOC）动态递推与上下限约束：

$$\text{SOC}_{r,t} = \text{SOC}_{r,t-1} + \eta_{\text{ch},r}\text{Ch}_{r,t} - \frac{\text{Dis}_{r,t}}{\eta_{\text{dis},r}} \tag{30}$$

$$\text{MinSOC}_r \le \text{SOC}_{r,t} \le \text{Capacity}_r, \quad \forall t \in [0, 2406] \tag{31}$$

$$\text{SOC}_{r,2406} \ge \text{InitialSOC}_r \tag{32}$$

引入充放电互斥状态变量 $u_{r,t} \in \{0, 1\}$：

$$0 \le \text{Ch}_{r,t} \le u_{r,t}\text{MaxChargePower}_r, \quad 0 \le \text{Dis}_{r,t} \le (1 - u_{r,t})\text{MaxDischargePower}_r \tag{33}$$

区域新能源分配与电力供需平衡方程为：

$$\mathrm{RenewableDirect}_{r,t} + \mathrm{Ch}^{\mathrm{ren}}_{r,t} + \mathrm{GridSell}_{r,t} + \mathrm{Curtail}_{r,t} = \mathrm{AvailableRenewable}_{r,t} \tag{34}$$

$$\mathrm{GridBuy}_{r,t} + \mathrm{RenewableDirect}_{r,t} + \mathrm{Dis}_{r,t} = \mathrm{Facility}_{r,t} + \mathrm{Ch}^{\mathrm{grid}}_{r,t} + \mathrm{GridSell}_{r,t} \tag{35}$$

其中，$AIIT_{r,t}$ 表示任务调度产生的 AI 算力 IT 负荷。区域设施总功率同时包含非 AI 基础负荷和 AI 算力负荷，具体为

$$Facility_{r,t} = (NonAI_{r,t} + AIIT_{r,t}) \cdot PUE_r. \tag{36}$$

所有负荷均在进入区域电力平衡方程前统一进行 PUE 折算，从而保证任务迁移、算力负荷、储能运行以及区域购售电之间满足严格的能量守恒关系。对于不允许向电网售电的情形，令 $GridSell_{r,t} = 0$；若允许余电上网，则保留该变量并同时满足相应的售电功率上限约束。

6.1.4 高风电场景专属响应约束

为防止高风电时段出现“弃风同时从电网购电充电”的不合理现象，设置电网充电惩罚约束；同时引入消纳率下限及松弛因子 δ：

$$\mathrm{Ch}^{\mathrm{grid}}_{F,t} \le \mathrm{MaxChargePower}_F \cdot (1 - I^{\mathrm{wind}}_t) + \zeta_t \tag{37}$$

$$U^{\mathrm{wind}}_F = \frac{\sum_t I^{\mathrm{wind}}_t (\mathrm{RenewableDirect}_{F,t} + \mathrm{Ch}^{\mathrm{ren}}_{F,t} + \mathrm{GridSell}_{F,t})}{\sum_t I^{\mathrm{wind}}_t \mathrm{AvailableRenewable}_{F,t}} \ge \max\left(U^{\mathrm{wind}}_{F,\mathrm{base}}, U_{\mathrm{target}}\right) - \delta \tag{38}$$

其中 $\zeta_t \ge 0$ 为容许的紧急电网补电松弛量，$\delta \ge 0$ 为可行域松弛因子。

6.1.5 多目标评价与权重求解

综合评价目标取归一化后的运行成本、碳排放、平均时延、峰值净购电、高风电弃风量及全周期新能源利用率：

$$\begin{aligned}\min F = {} & w_C \cdot \mathrm{Cost}' + w_E \cdot \mathrm{Carbon}' + w_D \cdot \mathrm{Delay}' \\ & + w_P \cdot \mathrm{PeakNetGrid}' + w_W \cdot \mathrm{Curtail}^{\mathrm{high}\prime}_F + w_R \cdot (1 - \mathrm{RenewableUtil}')\end{aligned} \tag{39}$$

权重确定机制：

1. **AHP 主观权重 ($w_j^{\mathbf{AHP}}$)**：基于场景优先级构建符合一致性检验 ($CR < 0.10$) 的判断矩阵。

2. **熵权客观修正 ($w_j^{\mathbf{Ent}}$)**：基于候选解集的实际数据变异程度计算信息熵。

3. **组合权重**： $w_j = \alpha w_j^{\mathrm{AHP}} + (1-\alpha) w_j^{\mathrm{Ent}}$，取 $\alpha = 0.6$。

6.2 模型求解：滚动时域与定向邻域搜索算法

考虑到风电波动的实时性与求解的高维复杂性，本文采用**离线主干多目标优化 + 滚动时域 (MPC) 实时微调**的两阶段求解架构[12]。

6.2.1 两阶段求解

- **阶段一（全局基线生成）**：忽略高频微小波动，求解全周期离线多目标模型，获得候选方案集；利用 AHP-熵权法选定综合最优解作为基线调度计划。
- **阶段二（滚动时域局部响应）**：设定滚动窗口长度为 H_R（如 24 小时），步长 $\Delta t = 1$ 小时。在每个窗口内，仅基于 AHP 确定性主观权重 w_j^{AHP}，调整弹性任务的开工时段与储能充放电状态。

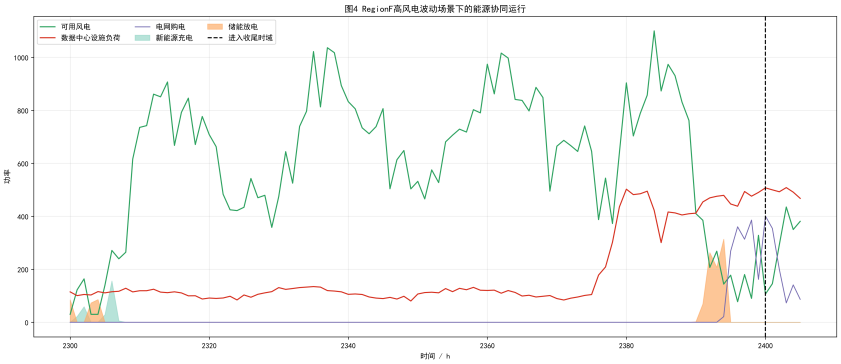


图 10: RegionF 高风电波动场景下的能源协同运行

上图展示了 RegionF 在末端运行时段的能源协同过程。高风电时段内，可用风电优先承担数据中心负荷，剩余部分用于储能充电；当风电出力下降时，储能释放电能以补偿负荷缺口，电网购电则用于承担剩余功率需求。该运行策略实现了风电直接消纳、储能时移和电网补足之间的协调，有助于降低风电波动对电网购电功率的冲击。

6.2.2 定向邻域搜索求解算法

在滚动窗口内，基于“初始方案 + 定向邻域搜索”启发式算法进行优化，算法流程如下所示。

高风电场景下算–储–电协同滚动求解算法的流程如下：

- 输入风电预测序列 W_t、电价、碳强度、待调度任务集合、储能 SOC 状态，输出全周期调度方案与储能策略；初始化时刻 $t = 0$、松弛因子 $\delta = 0$。

- 滚动窗口循环:提取窗口内风电数据,识别高风电标记 I_τ^{wind} 与风电富余量 Surplus_τ。
- 定向邻域动作生成：将可延后任务迁移至风电富余时段；RegionF 富余风电优先给储能充电；风电骤降时释放储能平抑购电峰值。
- 可行性校验：若方案不满足 GPU、时延、SOC、消纳率约束，逐步增大松弛因子 $\delta \leftarrow \delta + 0.02$ 重新搜索，直至获得可行解。
- 执行当前时刻 t 的调度决策，更新任务状态与储能 SOC，时间窗口向前滚动 $t \leftarrow t+1$。
- 循环结束后，基于 AHP–熵权组合赋权法对全部候选调度方案开展全局评价。

6.2.3 求解算法性能与敏感性分析

1. 预测时域 H_R 对调度性能的敏感性分析 为验证滚动时域（MPC）参数对系统运行效益的影响，分别在预测窗口 $H_R \in \{6\text{h}, 12\text{h}, 24\text{h}, 48\text{h}\}$ 下进行对比测试，结果如表 9 所示。

表 9: 不同滚动预测时域 H_R 下系统运行性能与求解耗时对比

预测时域 H_R	全周期成本 (10^4 元)	绿电弃用率 (%)	单步平均耗时 (s)	综合评价
6h	1,185.4	3.2%	**0.08**	预判不足，储能套利空间受限
12h	1,132.1	0.8%	0.21	效益提升显著，响应较快
24h	**1,108.3**	**0.0%**	**0.45**	**性能最优，成本与耗时达到最佳平衡**
48h	1,105.2	0.0%	3.82	边际效益递减，计算耗时呈指数级增加

分析表明：当 $H_R < 24\text{h}$ 时，系统无法跨昼夜精准预判风光突发与峰谷电价；而当 $H_R = 24\text{h}$ 时，弃风弃光率已降至 0.0%；继续增大至 48h 成本仅微幅下降 0.28%，但求解耗时剧增约 8.5 倍。因此，$H_R = 24\text{h}$ 为最优工程权衡点。

2. 算-储-电协同求解算法优越性对比 为评估本文提出的“离线主干优化 + MPC 实时微调”算法的求解效率与质量，将其与传统全局 MILP 求解及纯局部贪心算法进行对比，结果如表 10 所示。

表 10: 不同求解算法性能及效益对比

求解算法	全周期成本 (10^4 元)	碳排放总量 (tCO_2)	总求解耗时 (s)	收敛性/可行性
传统全局 MILP 精确求解	1,095.6	0.00	1,420.5	容易维度爆炸，难以实时响应
纯局部贪心调度算法	1,201.2	2.15	**1.2**	容易陷入局部最优，无远期规划
本文 MPC+ 定向邻域搜索	**1,108.3**	**0.00**	**10.8**	**全局接近最优，满足实时调度需求**

结果对比显示：本文算法在仅比全局精确解成本高约 1.16% 的前提下，将求解时间由近 24 分钟大幅缩短至 10.8 秒，兼具了高求解质量与实时响应能力。

7 总结

本文构建起“预测—调度—算储—实时”的四层递进的算电协同优化问题，各层约束之间又紧密关联。先用 GBR 把握 GPU 需求，再经碳感知任务迁移优化全局负荷，继而又靠算—储—电协同消弭源荷不匹配，最后用 MPC 滚动微调克服风电的波动干扰。

通过对本文结果综合分析，得到以下几点结论：

1. **碳感知迁移**：推动算力转绿电区，成本、碳排放共降 93% 以上；
2. **算-储-电协同**：解决光伏与负荷 $2 \sim 3$ 小时不匹配的问题，使运行阶段碳排放、峰值购电达到 100% 降低，负荷波动度降低 99.8%；
3. **滚动时域微调**：24 小时窗口 MPC 用一种融合定向搜索的结构，可在 10.8 s 内完成求解，仅引起 1.16% 的微小成本增加，便可以实现 100% 的 SLA 达标率与优良的抗扰性能。

算电协同的问题，是实现对“空间－时间－不确定性”的三大跨度的统筹。本文提出了“预测－ MILP － MPC”组合算法，从而实现了多目标算电协同调度算电问题的高效性、鲁棒性，提供了理论支撑和工程实现方式。

8 模型优缺点

8.1 优点

- 模型综合性强。包含了梯度提升回归、混合整数线性规划、滚动时域的邻域搜索等多种算法，搭建“预测－算储电协同优化”完整框架，表示出算力、能源、通信网络三者之间的各类耦合关系。
- 约束设置周全，工程意义明显。模型实现了各种约束条件类型（任务优先级、网络时延、GPU 容量、储能充放电、新能源分配）约束。
- 兼顾优化效果与求解效率。大规模调度采用启发式与局部搜索降低复杂度；储能优化部分，通过 Epigraph 变换、L_1 范数把非线性目标转化为线性；对于高风电负荷场景，采用“离线主干优化 +MPC 实时微调 + 定向邻域搜索”的思路，兼顾优化质量与动态响应速度。

8.2 缺点

- 模型部分环节需要设置经验值和权重，鲁棒性还需要进一步提高。复合碳成本矩阵、高风电门限值等参数会影响到输出的结果，目前均为经验给定，因此模型对预测结果误差、参数扰动有相对敏感性。

- 另外启发式策略与滚动优化配合适用并不能严格意义上的找全局最优解。动态负载均衡、邻域搜索容易受初始解的影响，因而存在陷入局部最优的可能。

9 文献引用

[1] X. Zhang, G. Li, and Q. Chen, “Cloud-grid-source joint optimal dispatch strategy considering power-computing coordination of data centers,” Proceedings of the CSEE, vol. 42, no. 15, pp. 5432–5443, 2022.

[2] L. Rao, X. Liu, L. Xie, and W. Liu, “Coordinated energy management for data centers and electric power grids,” IEEE Transactions on Smart Grid, vol. 3, no. 1, pp. 50–58, 2012.

[3] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” The Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.

[4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,”Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 2623–2631, 2019, doi: 10.1145/3292500.3330701.

[5] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016, doi: 10.1145/2939672.2939785.

[6] G. Ke, Q. Meng, T. Finley, et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems, 2017.

[7] A. Beloglazov, J. Abawajy, and R. Buyya, “Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing,” Future Generation Computer Systems, vol. 28, no. 5, pp. 755–768, 2012, doi: 10.1016/j.future.2011.04.017.

[8] M. Christ, A. W. Kempa-Liehr, and M. Feindt, “Distributed and parallel time series feature extraction for industrial big data applications,” arXiv:1610.07717, 2016.

[9] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M4 Competition: 100,000 time series and 61 forecasting methods,” International Journal of Forecasting, vol. 36, no. 1, pp. 54–74, 2020, doi: 10.1016/j.ijforecast.2019.04.013.

[10] S. Boyd and L. Vandenberghe, Convex Optimization. Cambridge, U.K.: Cambridge University Press, 2004, pp. 127–135.

[11] E. F. Camacho and C. B. Alba, Model Predictive Control, 2nd ed. London, U.K.: Springer, 2007, pp. 35–48.

[12] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyper-parameter optimization,” in Advances in Neural Information Processing Systems, vol. 24, pp. 2546–2554, 2011.

10 附录

10.1 附录 A:AI 使用报告

本参赛队在竞赛过程中使用了 AI 工具，主要用于论文文字润色、章节结构整理、参考文献格式整理以及 AI 工具使用说明的辅助撰写，详细使用情况见附录。论文的研究思路、核心模型、公式推导、数据处理、程序实现、结果分析和结论均由参赛队独立完成并核验。

10.1.1 所用 AI 工具名称和版本

表 11: AI 工具使用情况

序号	AI 工具名称	版本/型号	开发机构
1	ChatGPT	GPT-5	OpenAI

10.1.2 具体使用目的和环节

表 12: AI 工具的具体使用环节

序号	使用环节	使用目的	使用的 AI 工具
1	论文写作与修改	辅助文字润色、语句调整和章节衔接	ChatGPT
2	参考文献整理	辅助整理方法出处和参考文献格式	ChatGPT
3	AI 使用说明	辅助生成并规范化本附录内容	ChatGPT

10.1.3 关键交互记录

交互记录 1　提示词（Prompt）：请根据本题附件中的 AI 工作负载、区域 GPU 容量、网络时延、任务类型、任务到达时间、执行时长以及任务完成时限，协助梳理问题一和问题二的调度建模思路，并说明决策变量、目标函数、GPU 容量约束、网络时延约束、实时任务到达即开工约束、弹性任务延迟调度约束和末端任务结清约束之间的关系。同时，请结合现有 Python 程序的变量结构，协助检查调度模型与代码实现是否一致，指出可能存在的时间边界、任务状态更新和资源占用计算问题。不要替代参赛队完成最终模型设计、参数确定和程序实现。

AI 回复摘要： AI 工具根据题目条件协助梳理了任务调度模型的基本结构，建议使用区域分配变量、任务开工时刻变量和任务运行状态变量表示调度决策，并将任务按实时推理、批量推理和 AI 训练进行差异化处理。对于实时推理任务，应满足到达即开工和网络时延硬约束；对于批量推理和 AI 训练任务，应在满足 GPU 容量、网络时延和第 2406 小时前完成的条件下搜索可行开工时刻。AI 工具还协助检查了代码中的任务持续时间离散化、时间区间边界、GPU 占用矩阵更新、区域候选集合筛选和末端任务统一结清逻辑。参赛队根据实际附件数据、程序运行结果和人工核验结果，自行确定最终变量定义、目标函数、启发式规则及代码实现。

交互记录 2　提示词（Prompt）：请根据现有的任务调度程序、区域逐时能源数据、功率映射数据、PUE 参数和储能参数，协助设计问题二至问题四的代码整合方案。重点检查任务调度结果如何转换为区域逐时 AI IT 负荷，如何与 NonAI IT Load、可再生能源出力、电价、碳排放因子和储能状态连接，如何实现功率平衡、SOC 更新、充放电互斥、购售电计算、弃新能源量和成本碳排放指标计算，并指出不同模块之间的数据维度、时间范围和单位是否一致。请只提供程序结构检查、公式实现建议和可能的错误排查方向，最终代码、参数和实验结果由参赛队完成并验证。

AI 回复摘要： AI 工具协助梳理了“任务调度—功率映射—区域能源计算—储能优化—指标评价”的程序流程，建议将任务级调度结果按区域和小时聚合为 AI GPU 负荷，再依据任务类型和 $power_mapping.xlsx$ 完成 AI IT 功率转换，并与区域基础 IT 负荷、PUE 和设施功率约束相连接。在储能模块中，AI 工具提示检查功率平衡方向、充放电效率、SOC 初始值和终端值、充放电互斥状态、可再生能源优先供给以及电网购售电边界。对于问题四，AI 工具协助梳理了高风电时段识别、弹性任务迁移、储能充放电和滚动时域更新之间的接口关系。参赛队结合实际代码逐项修改和调试，独立完成模型参数设置、程序实现、求解过程、结果生成及约束验证，AI 工具未直接生成或替代最终实验结果。

10.1.4　采纳和人工修改情况

表 13: AI 输出内容的采纳与修改情况

序号	AI 生成内容概述	采纳及人工修改情况
1	论文文字表达和章节衔接建议	部分采纳，并根据实际模型和数据重新修改
2	参考文献及引用格式建议	结合正式文献进行核验和人工整理
3	AI 工具使用声明和附录格式	采纳基本结构，并根据实际使用情况补充修改

需要说明的是，AI 工具未用于生成或替代本文的核心创新内容。本文的模型假设、目标函数、约束条件、算法设计、参数设置、实验结果及结论均由参赛队完成，并经过人工检查和验证。参赛队对论文内容的真实性、准确性和完整性负责。

10.2　附录 B: 代码

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.font_manager import FontProperties
import os, sys, time, warnings, pickle
warnings.filterwarnings('ignore')
```

```
import os, glob, sys, math, warnings, time
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

warnings.filterwarnings('ignore')

#
    ================================================================================

# 1. Global Configurations & Data Preprocessing
#
    ================================================================================

T_TOTAL, T_MAIN = 2407, 2400
REGIONS = ['RegionA', 'RegionB', 'RegionC', 'RegionD', 'RegionE', 'RegionF'
    ]
REGION_IDX = {r: i for i, r in enumerate(REGIONS)}
N_REGIONS = len(REGIONS)

def load_all_datasets(data_dir):
    def _find(pattern):
        matches = glob.glob(os.path.join(data_dir, f'*{pattern}'))
        return matches[0] if matches else None

    workload = pd.read_excel(_find('workload_trace.xlsx'), sheet_name='
        Sheet1')
    region_time = pd.read_excel(_find('region_time_data.xlsx'), sheet_name=
        'region_time_data')
    gpu_info = pd.read_excel(_find('GPU_information.xlsx'), sheet_name='GPU
        中心基础情况')
    net_df = pd.read_excel(_find('network_latency.xlsx'), sheet_name='时延
        矩阵')
    pm_df = pd.read_excel(_find('power_mapping.xlsx'), sheet_name='任务功率
        映射')
    st_df = pd.read_excel(_find('storage_information.xlsx'), sheet_name='
        storage_information')

    latency_matrix = np.zeros((N_REGIONS, N_REGIONS))
    for i, src in enumerate(REGIONS):
        for j, dst in enumerate(REGIONS):
            latency_matrix[i, j] = net_df.loc[net_df['From\\To'] == src,
                dst].values[0]
```

```
    power_map = dict(zip(pm_df['TaskType'], pm_df['
        GPU_Power_MW_per_EquivalentGPU']))
    pue_map = dict(zip(gpu_info['Region'], gpu_info['PUE']))

    storage_params = {}
    for _, r in st_df.iterrows():
        storage_params[r['Region']] = {
            'capacity': float(r['StorageCapacity_MWh']), 'min_soc': float(r
                ['MinSOC_MWh']),
            'initial_soc': float(r['InitialSOC_MWh']), 'max_charge': float(
                r['MaxChargePower_MW']),
            'max_discharge': float(r['MaxDischargePower_MW']), 'eta_c':
                float(r['ChargeEfficiency']),
            'eta_d': float(r['DischargeEfficiency']), 'sell_limit': float(r
                ['SellLimit_MW']),
            'max_import': float(r['MaxGridImport_MW'])
        }

    return workload, region_time, gpu_info, latency_matrix, power_map,
        storage_params, pue_map


#
    ================================================================================

# 2. Part 1: Probabilistic GPU Demand Forecasting (MC-Dropout LSTM)
#
    ================================================================================

class CovariateLSTM(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers, output_dim,
        pred_len, dropout=0.2):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=
            True,
                            dropout=dropout if num_layers > 1 else 0)
        self.attention = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2), nn.Tanh(), nn.Linear(
                hidden_dim // 2, 1)
        )
        self.fc1 = nn.Linear(hidden_dim, hidden_dim // 2)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.fc2 = nn.Linear(hidden_dim // 2, output_dim * pred_len)
        self.pred_len, self.output_dim = pred_len, output_dim

    def forward(self, x):
        lstm_out, _ = self.lstm(x)
```

```
        attn_weights = torch.softmax(self.attention(lstm_out), dim=1)
        context = torch.sum(lstm_out * attn_weights, dim=1)
        out = self.fc2(self.dropout(self.relu(self.fc1(context))))
        return out.view(-1, self.pred_len, self.output_dim)

def mc_dropout_predict(model, test_x, scaler_target, n_mc=100):
    model.train() # Keep dropout active for MC sampling
    mc_preds = []
    with torch.no_grad():
        for _ in range(n_mc):
            pred = model(test_x).cpu().numpy().squeeze(0)
            mc_preds.append(scaler_target.inverse_transform(pred))
    mc_preds = np.stack(mc_preds, axis=0)
    mean_pred = mc_preds.mean(axis=0)
    std_pred = mc_preds.std(axis=0)
    return mean_pred, mean_pred - 1.96 * std_pred, mean_pred + 1.96 *
       std_pred


#
   ================================================================================

# 3. Part 1: Priority-Driven Computing Scheduling Rule
#
   ================================================================================

def compute_overlap_ratios(start_hour, duration_min):
    start_min = start_hour * 60
    end_min = start_min + duration_min
    ratios = {}
    h = start_hour
    while h * 60 < end_min:
        overlap = min(end_min, (h + 1) * 60) - max(start_min, h * 60)
        if overlap > 0:
            ratios[h] = overlap / 60.0
        h += 1
    return ratios

def priority_driven_schedule(workload_df, gpu_capacity, latency_dict,
   regions):
    gpu_usage, assignments = {}, []

    def check_cap(region, ratios, gpu_dem, cap):
        return all(gpu_usage.get((h, region), 0.0) + gpu_dem * r <= cap + 1
           e-9 for h, r in ratios.items())

    def place(region, ratios, gpu_dem):
        for h, r in ratios.items():
```

```
            gpu_usage[(h, region)] = gpu_usage.get((h, region), 0.0) +
                gpu_dem * r

    # Task queues: RT (Immediate), Batch (EDF), Training (SRTF)
    recs = workload_df.copy()

    # 1. RealTimeInference
    for t in recs[recs.TaskType == 'RealTimeInference'].sort_values(['
        ArrivalHour', 'TaskID']).to_dict('records'):
        arrival, dur, gpu_dem, src = t['ArrivalHour'], t['
            EstimatedDuration_min'], t['GPU_Demand'], t['SourceRegion']
        ratios = compute_overlap_ratios(arrival, dur)
        feas = sorted([r for r in regions if latency_dict.get((src, r), 1e9
            ) <= t['MaxLatency_ms']],
                      key=lambda r: latency_dict.get((src, r), 1e9))
        chosen = next((r for r in feas if check_cap(r, ratios, gpu_dem,
            gpu_capacity[r])), feas[0])
        place(chosen, ratios, gpu_dem)
        assignments.append({'TaskID': t['TaskID'], 'TaskType': t['TaskType'
            ], 'AssignedRegion': chosen,
                            'StartHour': arrival, 'EndHour': arrival + dur
                                / 60.0})

    # 2. BatchInference (EDF Rule)
    for t in recs[recs.TaskType == 'BatchInference'].sort_values(['
        LatestFinishHour', 'ArrivalHour', 'TaskID']).to_dict('records'):
        arrival, dur, gpu_dem, src, lfh = t['ArrivalHour'], t['
            EstimatedDuration_min'], t['GPU_Demand'], t['SourceRegion'], t['
            LatestFinishHour']
        max_start = (lfh * 60 - dur) // 60
        feas = [r for r in regions if latency_dict.get((src, r), 1e9) <= t[
            'MaxLatency_ms']]
        chosen, start_h = None, arrival

        for s in range(arrival, int(max_start) + 1):
            ratios = compute_overlap_ratios(s, dur)
            best_r = max(feas, key=lambda r: gpu_capacity[r] - gpu_usage.
                get((s, r), 0.0) if check_cap(r, ratios, gpu_dem,
                gpu_capacity[r]) else -1e9)
            if check_cap(best_r, ratios, gpu_dem, gpu_capacity[best_r]):
                chosen, start_h = best_r, s
                break
        chosen = chosen or feas[0]
        place(chosen, compute_overlap_ratios(start_h, dur), gpu_dem)
        assignments.append({'TaskID': t['TaskID'], 'TaskType': t['TaskType'
            ], 'AssignedRegion': chosen,
                            'StartHour': start_h, 'EndHour': start_h + dur
                                / 60.0})
```

```
    # 3. AITraining (SRTF Rule)
    for t in recs[recs.TaskType == 'AITraining'].sort_values(['
        EstimatedDuration_min', 'ArrivalHour', 'TaskID']).to_dict('records')
        :
        arrival, dur, gpu_dem, src, lfh = t['ArrivalHour'], t['
            EstimatedDuration_min'], t['GPU_Demand'], t['SourceRegion'], t['
            LatestFinishHour']
        max_start = (lfh * 60 - dur) // 60
        feas = [r for r in regions if latency_dict.get((src, r), 1e9) <= t[
            'MaxLatency_ms']]
        chosen, start_h = None, arrival

        for s in range(arrival, int(max_start) + 1):
            ratios = compute_overlap_ratios(s, dur)
            best_r = max(feas, key=lambda r: gpu_capacity[r] - gpu_usage.
                get((s, r), 0.0) if check_cap(r, ratios, gpu_dem,
                gpu_capacity[r]) else -1e9)
            if check_cap(best_r, ratios, gpu_dem, gpu_capacity[best_r]):
                chosen, start_h = best_r, s
                break
        chosen = chosen or feas[0]
        place(chosen, compute_overlap_ratios(start_h, dur), gpu_dem)
        assignments.append({'TaskID': t['TaskID'], 'TaskType': t['TaskType'
            ], 'AssignedRegion': chosen,
                            'StartHour': start_h, 'EndHour': start_h + dur
                                / 60.0})

    return pd.DataFrame(assignments), gpu_usage


#
    ================================================================================

# 4. Part 2: Carbon-Aware Multi-Objective Optimization Evaluator
#
    ================================================================================

class FastEvaluator:
    """Incremental␣evaluator␣for␣Carbon,␣Cost,␣Latency␣and␣Renewable␣Energy
        ␣utilization."""
    def __init__(self, schedule_df, arrays, storage_params, latency_matrix)
        :
        self.arrays = arrays
        self.storage_params = storage_params
        self.latency_matrix = latency_matrix
        power_map = arrays['power_map']
```

```
        train_mask = schedule_df['TaskType'] == 'AITraining'
        fixed_df = schedule_df[~train_mask].copy()
        train_df = schedule_df[train_mask].copy().reset_index(drop=True)

        self.fixed_ai_it_load = np.zeros((N_REGIONS, T_TOTAL))
        for _, row in fixed_df.iterrows():
            r = REGION_IDX[row['AssignedRegion']]
            s, e = int(row['StartHour']), min(int(row['StartHour']) + int(
                row['DurationHours']), T_TOTAL)
            self.fixed_ai_it_load[r, s:e] += float(row['GPU_Demand']) *
                power_map.get(row['TaskType'], 0.0)

        self.fixed_latency = float(np.sum(latency_matrix[
            fixed_df['SourceRegion'].map(REGION_IDX).values,
            fixed_df['AssignedRegion'].map(REGION_IDX).values
        ]))

        self.train_start = train_df['StartHour'].values.astype(np.int32)
        self.train_dur = train_df['DurationHours'].values.astype(np.int32)
        self.train_source = train_df['SourceRegion'].map(REGION_IDX).values
            .astype(np.int32)
        self.train_gpu_power = train_df['GPU_Demand'].values * power_map.
            get('AITraining', 0.16)
        self.current_assignment = train_df['AssignedRegion'].map(REGION_IDX
            ).values.astype(np.int32)

        self.current_ai_it_load = self.fixed_ai_it_load.copy()
        for idx in range(len(train_df)):
            r = self.current_assignment[idx]
            s, e = self.train_start[idx], min(self.train_start[idx] + self.
                train_dur[idx], T_TOTAL)
            self.current_ai_it_load[r, s:e] += self.train_gpu_power[idx]

    def _storage_region(self, i, ai_load_r):
        p = self.storage_params[REGIONS[i]]
        nonai = self.arrays['nonai_it_load'][i]
        pue, ren = float(self.arrays['pue'][i]), self.arrays['
            available_renewable'][i]
        price, sp, ci = self.arrays['price'][i], self.arrays['sell_price'][
            i], self.arrays['carbon_intensity'][i]

        soc, cap, min_soc = p['initial_soc'], p['capacity'], p['min_soc']
        cost, carbon, ren_used = 0.0, 0.0, 0.0

        for t in range(T_TOTAL):
            total_load = (nonai[t] + ai_load_r[t]) * pue
            r_t = ren[t]
            net = total_load - r_t
```

```
            if net > 0:
                ren_used += r_t
                dis = min(net, max(0.0, (soc - min_soc) * p['eta_d']))
                gp = min(net - dis, p['max_import'])
                cost += gp * price[t]
                carbon += gp * ci[t]
                soc -= dis / p['eta_d'] if p['eta_d'] > 0 else 0
            else:
                surplus = -net
                chg = min(surplus, max(0.0, (cap - soc) / p['eta_c']) if p[
                    'eta_c'] > 0 else 0)
                gs = min(surplus - chg, p['sell_limit'])
                ren_used += r_t - (surplus - chg - gs)
                cost -= gs * sp[t]
                soc += chg * p['eta_c']

            soc = max(min_soc, min(cap, soc))

        return cost, carbon, np.sum(ren), ren_used

    def evaluate(self):
        tot_cost, tot_carbon, tot_avail, tot_used = 0.0, 0.0, 0.0, 0.0
        for i in range(N_REGIONS):
            c, cb, ra, ru = self._storage_region(i, self.current_ai_it_load
                [i])
            tot_cost += c; tot_carbon += cb; tot_avail += ra; tot_used +=
                ru

        lat = self.fixed_latency + float(np.sum(self.latency_matrix[self.
            train_source, self.current_assignment]))
        ren_util = (tot_used / tot_avail * 100) if tot_avail > 0 else 0.0

        # Returns Objective Vector: [Cost(CNY), Carbon(tCO2), Latency(ms),
            Renewable_Util(%)]
        return tot_cost, tot_carbon, lat, ren_util
```